

Chapter 4

Decisions and Loops: Let Python Repeat the Work

Use if statements and loops to classify samples, count bases, and scan sequences.

EDOS learning idea

We will use biology and DNA examples because they are concrete, visual, and easy to experiment with. Type the examples yourself, change one thing, run again, and observe what changed.

What you will be able to do

- Write if/elif/else decisions.
- Use comparison operators.
- Repeat code with for loops.
- Use range() and enumerate().
- Count nucleotides and find motifs with loops.

Classroom rhythm

Read a small idea -> run the example -> change one value -> predict the result -> run again -> explain what happened in your own words.

4.1 if: let the program make a decision

An if statement runs code only when a condition is True.

```
gc_percent = 62

if gc_percent > 50:
    print('GC-rich sequence')
else:
    print('AT-rich sequence')
```

The colon and indentation matter. The indented lines belong to the condition.

4.2 Comparisons

Useful comparisons include == (equal), != (not equal), >, <, >=, and <=. A very common beginner mistake is using = when you mean ==. One equals sign assigns a value; two compare values.

```
base = 'G'
if base == 'G':
    print('This base is guanine')
```

4.3 for loops: the copy-paste machine

A for loop repeats a block once for each item in a collection.

```
dna = 'ATGC'
for base in dna:
    print(base)
```

Read this as: for each base in dna, print the base.

4.4 Counting DNA with a loop

```
dna = 'AGCTTTTCATTCTGAC'
a_count = 0

for base in dna:
    if base == 'A':
        a_count = a_count + 1

print('A count:', a_count)
```

Python's count() method is shorter for this exact job, but writing the loop teaches you how repeated logic works.

4.5 range() and enumerate()

```
for i in range(5):
    print(i)

dna = 'ATGCGT'
for position, base in enumerate(dna):
    print(position, base)
```

`range(5)` produces 0 through 4. `enumerate()` is excellent when you need both the position and the value.

4.6 Scan for a motif

Suppose you want to find every ATG in a short sequence. We can slide across the string one position at a time.

```
dna = 'CCATGAAATGCCC'
motif = 'ATG'

for i in range(len(dna) - len(motif) + 1):
    if dna[i:i+len(motif)] == motif:
        print('Found ATG at position', i)
```

This pattern - loop over positions, slice a window, compare - appears in many sequence-analysis ideas.

Try it yourself

These are practice prompts, not a test. If your code breaks, that is useful information. Read the error, change one thing, and try again.

1. Classify a sequence as short if `len(dna) < 20`, otherwise long.
2. Loop through a DNA string and count G bases manually.
3. Use `enumerate()` to print each base with its position.
4. Scan a sequence for the motif CG.
5. Loop through a list of gene names and print each one in lowercase.

Tiny debugging habit

When something fails, check spelling, quotation marks, brackets, colons, and indentation first. Then print intermediate values so you can see what Python sees.

Chapter summary

- `if/elif/else` adds decisions.
- Indentation defines blocks in Python.
- `for` loops repeat work.
- `range()` generates positions.
- `enumerate()` gives both index and value.
- Motif scanning combines loops, slicing, and comparisons.

Next step

Open Notebook 04 in EDOS Jupyter and run the matching exercises. Then continue to Chapter 5.

EDOS - Educational Desktop Operating Simulator
Copyright by Dr. Yash Munnalal Gupta